

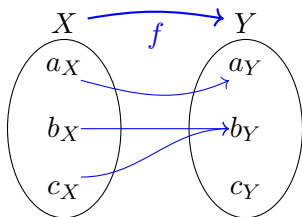
Set and Universal Constructions

Luca Roversi

- The Category **Set** of sets
 - Monomorphism
 - Epimorphism
- Universal constructions (in Haskell)
 - Terminal object
 - (Cartesian) Product

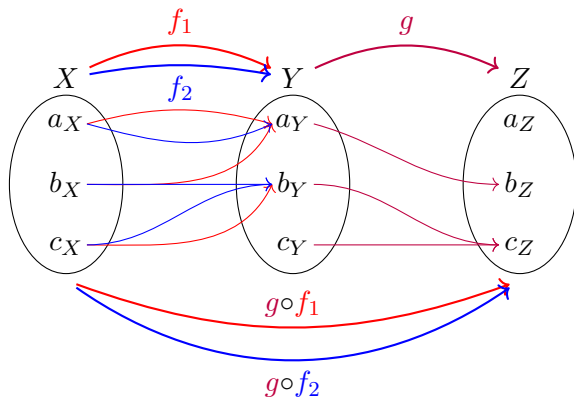
Category Set

Sets and functions among sets



- X is the *domain* of f
- Y is the *range* (co-domain) of f
- $\text{Img}(f) = \{y \mid \exists x \in X. y = f(x)\} \subseteq Y$
- f is *one-to-one/injective* $\Leftrightarrow$
 - $\forall x, w \in X. x \neq w \Rightarrow f(x) \neq f(w)$, or
 - $\forall x, w \in X. f(x) = f(w) \Rightarrow x = w$
- f is *onto/surjective* $\Leftrightarrow \forall y \in Y. \exists x \in X. y = f(x)$

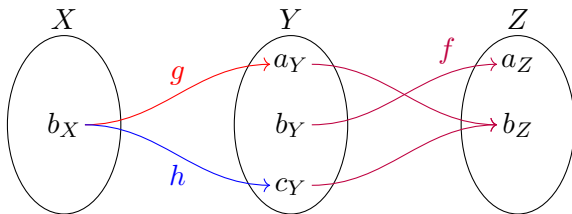
The Category Set of sets



- identities as expected: $\text{id}_X = \{(x, x) \mid x \in X\}, \dots$
- composition as expected:
 $g \circ f_1 = \{(x, g(f_1(x))) \mid x \in X, f_1(x) \in Y, g(f_1(x)) \in Z\}, \dots$

From “Injective function” to “Monomorphism”

f injective $\Leftrightarrow$ not(non injective) $\Leftrightarrow$ we cannot have:



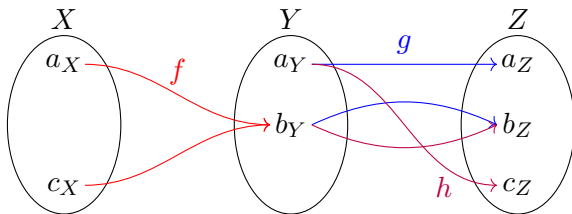
- $(\nexists g \ h, g \neq h \wedge f \circ g = f \circ h) \Leftrightarrow \overbrace{(\forall g \ h, f \circ g = f \circ h \Rightarrow g = h)}^{\text{canonical}}$
- f is “left erasable”

**f injective
iff**

one cannot say “ $g = h$ ” by composing f with g, h , but $g \neq h$

From “Surjective function” to “Epimorphism”

f surjective $\Leftrightarrow \text{not}(\text{non surjective}) \Leftrightarrow \text{we cannot have:}$



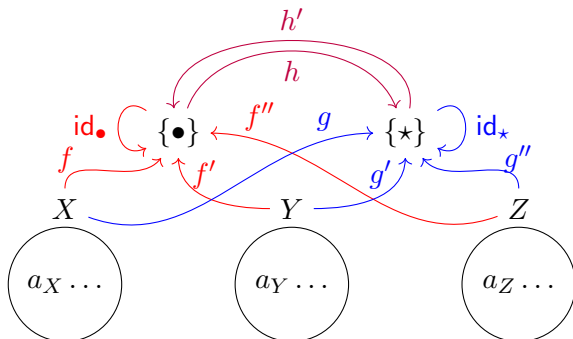
- $(\nexists g \ h, g \neq h \wedge g \circ f = h \circ f) \Leftrightarrow \overbrace{(\forall g \ h, g \circ f = h \circ f \Rightarrow g = h)}^{\text{canonical}}$
- f is “right erasable”

**f surjective
iff**

one cannot say “ $g = h$ ” by composing g, h with f , but $g \neq h$

Universal Constructions

Terminal object in Set: the singleton(s)



- How many functions $(A \setminus \{\star, \bullet\}) \rightarrow \{\bullet\}$ or $(A \setminus \{\star, \bullet\}) \rightarrow \{\star\}$?
- How many functions $\{\star\} \rightarrow \{\bullet\}$?
- How many functions $\{\bullet\} \rightarrow \{\star\}$?
- “Existence of a unique function” becomes a requirement for a universal construction

Terminal object in a category $\mathcal{C}$

Definition.

$T \in \text{obj}(\mathcal{C})$ is terminal iff

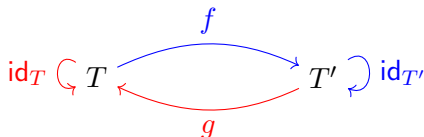
for every $X \in \text{obj}(\mathcal{C})$, there exists a *unique* arrow $X \rightarrow T$

Proposition.

Terminal objects are unique up to isomorphism

Proof.

- T is terminal for T' , and T' is terminal for T
- T, T' have their own identity
- The composition of the unique arrows $T \rightarrow T', T' \rightarrow T$ must be id_T
- The composition of the unique arrows $T' \rightarrow T, T \rightarrow T'$ must be $\text{id}_{T'}$



Haskell and the Terminal object

```
ghci> :i ()  
type () :: * -- it is type  
data () = () -- it is also a data: the unique element  
              -- in the set of elements with type ()
```

Exactly one function to `()` from every other type:

```
unit :: a -> () -- for every a  
unit _ = ()     -- forcefully a constant function
```

Bool is not Terminal in Haskell

```
ghci> :i Bool
type Bool :: *           -- it is a type
data Bool = False | True -- set with two elements
```

At least two functions to **Bool** from every other type:

```
oneFunction :: a -> Bool    -- for every a
oneFunction _ = true       -- a constant function
anotherFunction :: a -> Bool -- for every a
anotherFunction _ = false   -- another constant function
```

Cartesian Product in Set (via Haskell syntax)

Cartesian Product *pattern*:

- Set $A \times B$ of pairs
- *Projections* connecting $A \times B$ to A, B

In Haskell, if $p :: (a, b)$:

```
ghci> :i fst
fst :: (a, b) -> a
ghci> :i snd
snd :: (a, b) -> b
ghci> (fst p, snd p) == p
True
```

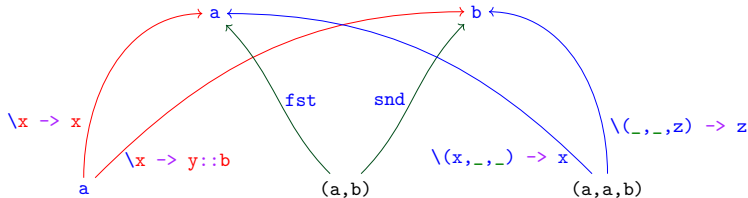
Category diagram: $\text{fst } p \in a \xleftarrow{\text{fst}} p \in (a, b) \xrightarrow{\text{snd}} b \ni \text{snd } p$

Why (a,b) Cartesian Product of a and b?

① Why not using:

- simply a
- or the more complex (a, a, b)

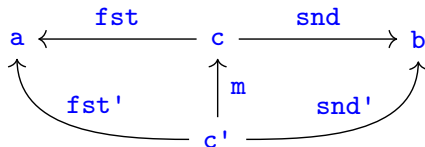
as Product of a and b, instead of (a,b)?



Criterion to decide the Cartesian Product of a and b

For every Cartesian Product *candidate* $a \xleftarrow{\text{fst}'} c' \xrightarrow{\text{snd}'} b$
the actual (cartesian) product c of a and b **must be better** than c' , i.e.:

- **there exists a unique** arrow $c' \xrightarrow{m} c$
- s.t. projections fst , snd are better than fst' , snd' , i.e.:

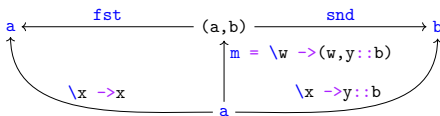


commutes, which means:

- $\text{fst}' = \text{fst} \circ m$, i.e. fst' is *reconstructed* by m and fst
- $\text{snd}' = \text{snd} \circ m$, i.e. snd' is *reconstructed* by m and snd
- or, equivalently:
 - m factorizes fst' and snd' using fst and snd

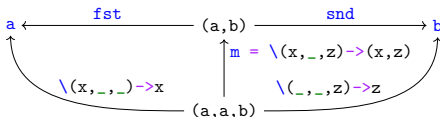
$((a,b),fst,snd)$ better than other proposals

- $((a,b),fst,snd)$ better than $(a,\backslash x \rightarrow x, \backslash x \rightarrow y :: b)$



because $m = \backslash w \rightarrow (w, y :: b)$ factorizes $\backslash x \rightarrow x$ and $\backslash x \rightarrow y :: b$:

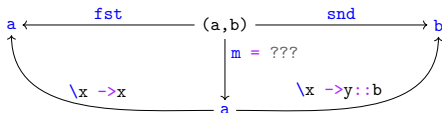
- $(\backslash(x, _) \rightarrow x)(m\ z) == (\backslash(x, _) \rightarrow x)(z, y :: b) == z$
- $(\backslash(_, y) \rightarrow y)(m\ z) == (\backslash(_, y) \rightarrow y)(z, y :: b) == y$
- $((a,b),fst,snd)$ better than $((a,a,b),\backslash(x,_,_) \rightarrow x, \backslash(_,_,z) \rightarrow z)$



because $m = \backslash(x,_,z) \rightarrow (x,z)$ factorizes $\backslash(x,_,_) \rightarrow x$, $\backslash(_,_,z) \rightarrow z$:

- $(\backslash(x,_) \rightarrow x)(m\ (x,y,z)) == (\backslash(x,_) \rightarrow x)(x,z) == x$
- $(\backslash(_,y) \rightarrow y)(m\ (x,y,z)) == (\backslash(_,y) \rightarrow y)(x,z) == z$

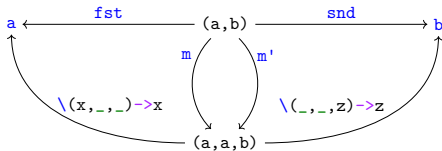
$(a, \lambda x. \rightarrow x, \lambda x. \rightarrow y :: b)$ worst than $((a, b), \text{fst}, \text{snd})$



$m = \lambda (w, _) \rightarrow w$ is the unique possible choice s.t.:

- its output matches the type
- **it can** factorize fst
- **but it cannot** factorize snd

$((a, a, b), \backslash(x, _, _) \rightarrow x, \backslash(_, _, z) \rightarrow z)$ worst than $((a, b), \text{fst}, \text{snd})$



The factorization of fst is not unique:

- $m = \backslash(x, y) \rightarrow (x, x, y)$
- $m' = \backslash(x, y) \rightarrow (x, n :: a, y)$, for any $n :: a$

Product in a category $\mathcal{C}$

A Product in a category $\mathcal{C}$ is a triple $(A \times B, \pi_l, \pi_r)$ such that:

- A, B are in $\text{obj}(\mathcal{C})$
- π_l is in $\mathcal{C}(A \times B, A)$, and π_r is in $\mathcal{C}(A \times B, B)$
- for every C in $\text{obj}(\mathcal{C})$, a *unique* m in $\mathcal{C}(C, A \times B)$ exists s.t.

A commutative diagram illustrating the universal property of the product. At the top, three objects are arranged horizontally: A on the left, $A \times B$ in the center, and B on the right. A solid arrow labeled π_l points from $A \times B$ to A , and another solid arrow labeled π_r points from $A \times B$ to B . Below these, the object C is positioned. A dashed vertical arrow labeled m points from C up to $A \times B$. Two curved solid arrows originate from C : one labeled f curves from C to A , and another labeled g curves from C to B . The diagram shows that the composition of m with π_l (via f) and with π_r (via g) yields the same result as the direct maps from C .

commutes, i.e.:

- $f = \pi_l \circ m$
- $g = \pi_r \circ m$

Intuitive summary and Exercises

What Universal constructions can be compared to?

- A query for a web-search engine is like “searching for a pattern”
- A *general* query gives large *recall*, i.e. lots of hits:
 - some are relevant
 - some others are not
- A *refined* query increases precision: less irrelevant hits
- Eventually, the top of the list is the best fit according to some “measure”

Analogy.

- Terminal/Initial object, Cartesian Product are interesting patterns
- Useful categories contain a lot of hits of *potentially* interesting patterns
- *Universal construction* defines patterns by relating objects/arrows:
 X is the best fit if, for every other Y , there is a *unique* Z such that X, Y, Z are related ... and involve decomposition
- *Universal construction* “measures” the best fitting pattern

Universal constructions as “best fitting” patterns

- Terminal object.

The universal definition says when an object A is “more terminal” than any other object B

- Product.

The universal definition says when an object A is “more a product” than any other object B

Unicity and Isomorphism

- Is the Product of $\mathfrak{a}$ and $\mathfrak{b}$ unique up to isomorphism?
- Are $(\mathfrak{a}, ())$ and $\mathfrak{a}$ isomorphic?

Product in Haskell

Are the following triples:

- $((\text{Bool}, \text{Int}, \text{Bool}), \lambda(x,y,z) \rightarrow y, \lambda(x,y,z) \rightarrow z)$
- $((\text{Bool}, \text{Int}, \text{Bool}), \lambda(x,y,z) \rightarrow y, \lambda(x,y,z) \rightarrow x)$
- $((\text{Int}, \text{Bool}), \lambda(x,y) \rightarrow x + 1, \lambda(x,y) \rightarrow y)$
- $((\text{Int}, \text{Bool}), \lambda(x,y) \rightarrow x, \lambda(x,y) \rightarrow \text{not } y)$

Products between the Haskell types `Int` and `Bool`?

Hint.

- Recall the definition of Product as universal construction
- Which conditions of Products are violated?



Milewski, B. and I. Tabachnik (2019). *Category Theory for Programmers*.

<https://bartoszmilewski.com/2014/10/28/category-theory-for-programmers-the-preface/>. Lightning Source UK. ISBN: 9780464243878. URL:

<https://books.google.it/books?id=hA55yQEACAAJ>.